SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Case Study

Course Code:	**ITA05**	**Course Title:**	**Computer Vision**
CO Assessed:	**CO3 – Precision (accurate feature extraction & analysis) (BL3)**	**Assessment:**	**Assessment Tool 2 – Case Study**
Weightage:	**40% of CIA**	**Total Marks:**	**100**
CO3 Statement:	*Analyze and extract image features using detection and matching techniques for effective image representation and comparison. (BL4)*		
Student Name:	Kaviya v	**Reg. No.:**	192421022
Section / Batch:	Third Year/ B.Tech IT	**Date:**	11/08/26

Code of Conduct

I ___Kaviya V (192421022)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _Kaviya V___

Criteria	Weigh tage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly	
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts	
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning	
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided	
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand	

Preprocessing Impact on Edge Detection

Scenario: A system performs edge detection directly on raw images containing noise and uneven illumination, resulting in fragmented and inaccurate edges. The problem is to analyze how preprocessing can improve edge detection accuracy and edge continuity.

1. Problem Analysis

Raw images are not always suitable for direct edge detection because:

- Noise creates unwanted intensity changes that can be detected as false edges.
- Uneven illumination causes different regions of the same object to have different brightness levels.
- Weak edges may disappear because of poor contrast.
- True boundaries can become broken or fragmented.
- Direct edge detection may therefore produce both false edges and missing edges.

Hence, preprocessing should be performed before edge detection.

2. Suitable Preprocessing Techniques

A suitable preprocessing pipeline is:

Raw Image
↓
Noise Reduction
↓
Illumination / Contrast Correction
↓
Grayscale Image
↓
Edge Detection
↓
Continuous and Accurate Edges

Step 1: Noise Reduction
A Gaussian filter can be applied before edge detection.
It smooths small intensity variations caused by noise.

$$I_{smooth} = G_\sigma * I$$

where (G sigma) is the Gaussian kernel.

Effect:

- Reduces random noise.
- Prevents noise pixels from being detected as edges.
- Produces cleaner edge maps.

For salt-and-pepper noise, a median filter can be more suitable because it removes isolated noisy pixels while preserving edges.

3. Uneven Illumination Correction

Uneven lighting can make the same object appear bright in one region and dark in another. A suitable approach is adaptive/local contrast enhancement or adaptive thresholding, depending on the subsequent task.

For contrast enhancement, CLAHE (Contrast Limited Adaptive Histogram Equalization) can be used.

Why CLAHE?
CLAHE improves contrast locally rather than applying the same enhancement to the entire image.

Advantages:

- Improves visibility of weak boundaries.
- Handles different brightness levels in different regions.
- Limits excessive contrast enhancement.
- Helps the edge detector identify previously weak boundaries.

4. Edge Detection

After preprocessing, the Canny Edge Detector can be applied.

Canny workflow:

Preprocessed Image
↓
Gradient Calculation
↓
Non-Maximum Suppression
↓
Double Thresholding
↓
Edge Tracking by Hysteresis
↓
Final Edge Map

Canny is particularly appropriate because it already includes Gaussian smoothing as part of its standard pipeline and uses non-maximum suppression and hysteresis to produce thin, connected edges.

5. Comparison: Without vs With Preprocessing

Aspect	Without Preprocessing	With Preprocessing
Noise	Creates false edges	Noise is reduced
Illumination	Produces inconsistent edges	Local contrast is improved
Weak boundaries	May be missed	More visible
Edge continuity	Fragmented	More continuous
False edges	High	Reduced
Detection accuracy	Lower	Higher

6. Why Preprocessing Improves Edge Continuity

Edge detection is based mainly on intensity changes. Noise and uneven illumination create unwanted intensity changes that can interfere with the actual object boundaries.

After preprocessing:

1. Noise-related intensity variations are reduced.
2. Important boundaries become clearer.
3. Weak edges become more distinguishable.
4. Canny can connect valid edge pixels using hysteresis.
5. The resulting boundaries become thinner, cleaner, and more continuous.

Therefore:

$$\boxed{\text{Preprocessing} \rightarrow \text{Cleaner Input} \rightarrow \text{Better Edge Detection}}$$

7. Recommended Approach

For this scenario, the recommended pipeline is:

Gaussian/Median Filtering → CLAHE → Canny Edge Detection

Justification

- Gaussian Filter: suitable for general random noise.
- Median Filter: better when salt-and-pepper noise is present.
- CLAHE: improves local contrast under uneven illumination.
- Canny: provides accurate and relatively continuous edge detection.

The exact preprocessing filter should depend on the type of noise present. Using excessive smoothing should be avoided because it can remove genuine fine edges.

Conclusion:

The fragmented edges are mainly caused by noise and uneven illumination in the raw image. Applying appropriate preprocessing before edge detection reduces false responses, improves local contrast, and strengthens weak boundaries. A combination of noise filtering, local contrast enhancement such as CLAHE, and Canny edge detection can therefore produce cleaner, more accurate, and more continuous object boundaries.